AI ENGINEERING ROADMAP · JUNIOR PYTHON DEV · 2026

FROM API CALLS TO AUTONOMOUS AGENTS

Updated Edition — now includes Claude Code workflows, Skills, and AI IDE translation for Phases 2–4

4 PHASES	20 WEEKS	4 PROJECTS	8+ COURSES	+NEW IDE SKILLS

01 WEEKS 1–3
API Foundations & Prompt Engineering
→ *Smart Expense Tracker*

02 WEEKS 4–7
Agents & Agentic Workflows
→ *Natural Language Calendar*

03 WEEKS 8–13
Claude Code, Multi-Agent & RAG
→ *AI Support Agent with RAG*

04 WEEKS 14–20
Model Context Protocol
→ *Life Admin MCP Hub*

PHASE 01 · WEEKS 1–3 — COMPLETED ✓

API Foundations & Prompt Engineering

Phase 1 is complete. Content unchanged. Project: Smart Expense Tracker.

PHASE 02 · WEEKS 4–7

Agents & Agentic Workflows

Build systems that reason, plan, and take multi-step actions. This is where you go from calling an API to building something that acts.

COURSES & RESOURCES

Agentic AI Docs

Anthropic's official agent guide: tool loops, human-in-the-loop, error handling, and when to use fully vs. semi-autonomous agents. Read this before building — it saves hours of confusion.

docs.anthropic.com/agents | Core Reading

Tool Use Deep Dive

JSON Schema tool definitions, parallel tool calls, and result handling. Think of tools as typed API endpoints for Claude — your backend experience makes this click faster than most.

docs.anthropic.com | Tool Use

Claude Agent SDK Course

Covers managed agent infrastructure — stateful sessions, persistent event history, and deploying agents without managing conversation state yourself. Significant productivity gain.

anthropic.skilljar.com | Free | Certificate

CLAUDE CODE & AI IDE SKILLS — NEW

■ Building Agent Workflows with Claude Code

Before writing agents from scratch, learn to scaffold and iterate using Claude Code in your terminal. Use it to generate tool definitions, debug tool-loop logic, and refactor agent code — then translate the same patterns into Cursor or Windsurf once you understand the mechanics.

WHAT TO BUILD WITH CLAUDE CODE THIS PHASE

- ■ Write a `CLAUDE.md` in your calendar-agent repo — describe the tool schema, the loop pattern, and coding conventions. Claude Code will follow it autonomously.
- ■ Use Claude Code to scaffold all 4 Google Calendar tool wrappers from a single plain-English description. Review the output, iterate, commit.
- ■ Practice the *Plan Mode → approve → execute* loop for destructive operations (create/delete calendar events). This mirrors the human-in-the-loop pattern you are learning.

TRANSLATING TO CURSOR / WINDSURF

Once the agent is working via Claude Code, open the same repo in Cursor or Windsurf. The agent architecture, tool schemas, and CLAUDE.md conventions all carry over directly. AI IDE = Claude Code's UX, running on the same underlying models. The mental model you build here transfers completely.

SKILLS PRACTICED

CLAUDE.md authoring	Plan Mode	Tool scaffold	Agentic iteration	IDE translation

PHASE 2 PROJECT

PROJECT 02

Natural Language Calendar Agent

Talk to your Google Calendar in plain English — schedule, reschedule, find free slots, and get daily briefings

Google Calendar API (free)	~4 hrs setup	Real agent tool loop

■ What You're Actually Learning

■ **Tool loop in real conditions** — 'book me a slot when I'm free' requires Claude to chain `get_events`, `find_free_slots`, then `create_event` — 3 calls in one request.

■ **Ambiguity handling** — write tool descriptions so Claude resolves fuzzy inputs before acting.

■ **Human-in-the-loop** — confirmation step for destructive actions. A critical production pattern.

Multi-step tool chaining	Tool loop pattern	External API integration	Ambiguity resolution	Human

■ Stretch Goals

■ Morning briefing — daily 8am message summarising events, travel time, suggested lunch window.

■ Add `get_weather(date)` via the free Open-Meteo API.

■ **Claude Code challenge:** use Claude Code to write a `CLAUDE.md` Skills file for this project, then direct it to add the Telegram bot integration without typing a line of code yourself.

PHASE 03 · WEEKS 8–13

Claude Code, Multi-Agent & RAG

Use AI in your terminal, orchestrate specialised agents, and give Claude long-term memory over large private knowledge bases.

COURSES & RESOURCES

Claude Code in Action

Official Anthropic course on Coursera & Skilljar. CLAUDE.md config, Plan Mode, hooks, custom slash commands, sub-agents, GitHub integration. The hands-on terminal course.

coursera.org	Free	Certificate

Introduction to Agent Skills

Build reusable markdown Skills for Claude Code — custom instructions that trigger automatically for the right tasks. Essential for scaling your personal development workflow.

anthropic.skilljar.com	Free	Certificate

Multi-Agent Architecture Docs

Orchestrator/subagent patterns, parallelisation, and specialised agent networks. Maps directly to your backend microservices mental model — agents are services that decide what to call next.

docs.anthropic.com | Architecture

Embeddings & RAG — Anthropic Cookbook

Anthropic's official RAG recipes: chunking strategies, embedding models, vector DB integration, and retrieval-augmented generation patterns.

github.com/anthropics/anthropic-cookbook | Python | Free

CONCEPT — WHAT IS RAG?

Retrieval-Augmented Generation (RAG)

How to give Claude memory over thousands of documents it has never seen

■ THE PROBLEM RAG SOLVES

Claude has a context window limit. You can't paste 500 documents into every prompt. RAG lets Claude answer questions about a large private knowledge base by only retrieving the relevant chunks at query time.

■ HOW IT WORKS (3 STEPS)

■ **Chunk & Embed** — split docs into chunks, convert to vectors, store in ChromaDB or pgvector.
■ **Retrieve** — embed the query, find most similar chunks using cosine similarity.
■ **Generate** — inject retrieved chunks into Claude's context. Claude answers with your private knowledge.

■ KEY TOOLS

■ **Embedding model** — voyage-ai (Anthropic partner, higher quality)
■ **Vector DB** — ChromaDB (local) or Supabase pgvector (cloud)
■ **Chunking** — fixed-size vs semantic; chunk size is the main tuning lever

PHASE 3 PROJECT

PROJECT 03

AI Support Agent with RAG

A smart Q&A chatbot for any product or service — answers from uploaded docs, FAQs, and manuals using RAG, built with Claude Code

RAG + Multi-agent + Claude Code | ChromaDB (local) | Streamlit UI | ~5 hrs setup

■ Agent Architecture

■ **Ingestion agent** — triggered once: reads PDFs, chunks (500 tokens / 50 overlap), embeds each chunk, stores in ChromaDB with source metadata.

■ **Retrieval agent** — on each query: embeds the question, similarity search, returns top-3 chunks with confidence scores. Applies score threshold.

■ **Answer agent (orchestrator)** — receives chunks + question, constructs a grounded prompt: *"Answer using ONLY the context below. If the answer isn't there, say so."* No hallucinations.

■ What You're Actually Learning

■ RAG pipeline end-to-end — chunk, embed, store, retrieve, generate. No LangChain magic hiding the mechanics.

■ Grounding prompts — system prompts that force Claude to stay within retrieved context and admit when it doesn't know.

■ Retrieval quality tuning — chunk size, overlap, score thresholds. The main RAG engineering skill.

■ Claude Code as dev tool — scaffold a multi-file project by directing Claude Code, not typing every line yourself.

RAG pipeline	Chunking strategies	Vector embeddings	Similarity search	Grounding prompts	Multi-ag

CLAUDE CODE SKILLS & WORKFLOWS — NEW

■ Building and Using Claude Code Skills for RAG Projects

Phase 3 is the natural place to master **Claude Code Skills** — reusable markdown instruction files that teach Claude Code the conventions of your project so it can scaffold, extend, and refactor autonomously. This is the bridge between Claude Code and production AI IDE workflows in Cursor/Windsurf.

CREATING YOUR FIRST SKILL FILE

Create `.claude/skills/rag-pipeline.md` in your project root. Inside, describe: your chunking strategy and why, the ChromaDB collection schema, how the 3-agent architecture connects, and coding conventions. Claude Code reads this automatically and applies it to every task in the project.

CUSTOM SLASH COMMANDS — BUILD THESE

■ `/ingest [path]` — re-run the ingestion agent on new PDFs

■ `/test-retrieval [query]` — run a retrieval test and show confidence scores

■ `/tune-chunks [size] [overlap]` — re-chunk and re-embed with new params, compare results

SUB-AGENTS IN CLAUDE CODE

Use Claude Code's sub-agent feature to mirror your RAG architecture directly: one sub-agent handles ingestion, one handles retrieval, the orchestrator agent calls both. This is the same multi-agent pattern from the API — just driven from the terminal.

TRANSLATING TO CURSOR / WINDSURF

Your `CLAUDE.md` and Skills files become **Cursor Rules** (`.cursorrules`) or Windsurf's **Cascade context files**. The concept is identical: give the IDE-embedded AI your project conventions so it codes to your architecture. Copy the content, rename the file — done.

Skills file authoring	Custom slash commands	Sub-agents	Cursor Rules	Windsurf Cascade	IDE transla

■ Stretch Goals

- ■ Hybrid search — combine vector similarity with BM25 keyword search.
- ■ Chat history — multi-turn session memory so follow-up questions work naturally.
- ■ Swap ChromaDB for Supabase pgvector — same pattern, now cloud-hosted, production-ready.
- ■ **Claude Code challenge:** write a Skills file that documents the RAG pipeline, then ask Claude Code to add hybrid search without any manual code. Review and commit only.

PHASE 04 · WEEKS 14–20 · CAPSTONE

Model Context Protocol (MCP)

Connect Claude to every tool in your life. This is where everything you built snaps together into one interface.

COURSES & RESOURCES

Introduction to MCP

Anthropic Academy's MCP course. The three primitives: Tools (model-controlled), Resources (app-controlled), Prompts (user-controlled). Build servers with the Python SDK using decorators.

anthropic.skilljar.com	Free	Certificate	Requires Python

MCP: Advanced Topics

Production MCP patterns: sampling, notifications, file system access, transport mechanisms. Covers the MCP Inspector for debugging. The jump from intro to production-grade servers.

anthropic.skilljar.com	Free	Certificate

MCP Python SDK Docs

Official spec and Python implementation at modelcontextprotocol.io. Transport layers (stdio, HTTP/SSE), authentication, real-world examples. Your primary reference during Phase 4.

modelcontextprotocol.io	Reference

PHASE 4 CAPSTONE PROJECT

CAPSTONE PROJECT 04

Life Admin MCP Hub

One MCP server connecting Claude to your calendar, expenses, and finances — control everything from a single conversation in Claude Desktop

MCP Python SDK	Connects all 3 prior projects	Claude Desktop + Claude Code	~6 hrs setup

■ The Three MCP Primitives You Build

■ **Tools** (Claude decides when to call): `log_expense()`, `query_expenses()`, `get_calendar()`, `create_event()`, `get_finance_summary()`, `add_home_task()`

■ **Resources** (browsable context): A `life://dashboard` resource — live snapshot of this week's calendar, month's budget, open tasks.

■ **Prompts** (one-click templates): A `weekly_review` prompt that pre-loads all three data sources and generates your weekly life summary. Every Sunday, one click.

■ What You're Actually Learning

■ **MCP architecture end-to-end** — feel the real difference between Tool (Claude-initiated), Resource (context you provide), and Prompt (user-initiated template).

■ **Cross-domain reasoning** — the Hub knows money AND time simultaneously, enabling queries neither prior agent alone could answer.

■ **Protocol thinking** — MCP is a standard. Once built, any MCP-compatible client (Claude Code, Claude Desktop, Cursor, Windsurf) connects without changes.

MCP server build	Tools primitive	Resources primitive	Prompts primitive	Cross-domain reasoning

CLAUDE CODE, AI IDE CONNECTORS & AGENT SYSTEMS — NEW

■ Connecting Your MCP Hub to Claude Code, Cursor, and Windsurf

Phase 4 is where MCP stops being a Claude Desktop feature and becomes a universal protocol. Your MCP server is client-agnostic by design — any MCP-compatible AI can call it. This phase teaches you to wire it into Claude Code, then translate the same server to work inside Cursor and Windsurf.

TASK 1 — USE YOUR MCP SERVER FROM CLAUDE CODE

- ■ Register your MCP server in Claude Code's config (`~/.claude/claude.json`). Claude Code uses stdio transport; your server is already stdio-compatible.
- ■ Write a CLAUDE.md Skills file for your hub: document every tool, its inputs, and when Claude Code should call it autonomously.
- ■ Create custom slash commands: `/weekly-review`, `/expense-summary`, `/free-slots [day]` — each triggers the relevant MCP tools automatically.

TASK 2 — BUILD AN MCP-CONNECTED WORKFLOW IN CURSOR

- ■ Cursor supports MCP servers natively (Settings → MCP). Add your server's stdio command — same config as Claude Desktop.
- ■ In Cursor's Composer (agent mode), ask it to build a new feature for your MCP Hub — e.g. a `get_tasks()` tool. Cursor calls your MCP server to inspect existing tools, then writes the new one to match.
- ■ Translate your CLAUDE.md Skills file into a `.cursorrules` file. Content is identical — the file name and location differ.

TASK 3 — WINDSURF CASCADES AS AGENT WORKFLOWS

- ■ Add your MCP server to Windsurf via its MCP config panel. Windsurf's Cascade (multi-step agentic mode) can call your tools across a long task chain.
- ■ Experiment: give Cascade a cross-domain task — *"I've overspent on food. Find a free evening this week to cook at home and block it."* It calls your expense tool, then your calendar tool — the same logic you built in the hub.
- ■ Write a Windsurf context file (equivalent to CLAUDE.md) describing the hub architecture so Cascade codes to your conventions when extending it.

SKILL EQUIVALENCES ACROSS TOOLS

Concept	Claude Code	Cursor	Windsurf
Project instructions	`CLAUDE.md`	`.cursorrules`	`.windsurfrules`
Reusable task patterns	Skills files	Cursor Rules (scoped)	Cascade context files
External tools (MCP)	claude.json MCP config	Settings → MCP	MCP config panel
Agentic mode	Claude Code (default)	Composer (Agent mode)	Cascade
One-click workflows	Custom slash commands	Prompt templates	Cascade workflows

☐ Stretch Goals

- ☐ `send_whatsapp(message)` via WhatsApp Cloud API — morning briefing to your phone.
- ☐ Home task manager — `add_task()`, `complete_task()`, `get_tasks()` backed by Supabase.
- ■ **Claude Code challenge:** write a CLAUDE.md Skills file that documents the entire hub. Direct Claude Code to add a new MCP tool end-to-end without typing a line yourself. Then replicate the same in Cursor's Composer to feel the difference.
- ☐ Build a minimal second MCP server (e.g. a notes server) and connect it alongside the hub in Claude Code, Cursor, and Windsurf. Practice multi-server MCP config.

MASTER RESOURCE LIST

Anthropic Academy
All 13 free official courses. Start here.
anthropic.skilljar.com

Anthropic Docs
Full API reference, agents, tool use, prompt engineering.
docs.anthropic.com

Claude Code in Action
Free Coursera course by Anthropic on Claude Code.
coursera.org

anthropics/courses
Official Jupyter notebooks, 5 sets. Clone and run locally.
github.com/anthropics/courses

MCP Spec & Python SDK
Official protocol reference and Python SDK.
modelcontextprotocol.io

Claude API Platform Docs
Models, endpoints, parameters, code examples.
platform.claude.com

Cursor Docs — Rules
How to write .cursorrules for project-level AI conventions.
docs.cursor.com/context/rules

Windsurf Docs — Cascade
Cascade workflows, context files, and MCP integration.
docs.codeium.com/windsurf

Expense Tracker → Calendar Agent → RAG Support Agent → MCP Hub · Each project feeds the next

2026 · Anthropic Academy · All courses free · Claude Code → Cursor / Windsurf translation included